

Algorithms Final Project - Gale-Shapley Study Room Assignment

Problem statement and real-world use case. A school needs to assign study groups to limited study rooms. Each group ranks rooms by convenience, equipment, or capacity, while each room ranks groups by priority such as reservation status, group size fit, or accessibility need. The goal is a complete assignment with no blocking pair: no group and room would both prefer each other over their current matches. This project implements Gale-Shapley stable matching to create fair, predictable room assignments.

Core algorithm description

Gale-Shapley repeatedly lets unmatched groups propose rooms in preference order. A free room is accepted immediately. A matched room compares the proposing group with its current group using a precomputed ranking table. If the new group is preferred, the room switches and the displaced group becomes free; otherwise, the proposing group remains free and later proposes to its next room. The loop ends when every group is matched. Because each group proposes to each room at most once, the algorithm terminates and produces a stable matching.

```
def gale_shapley(group_pref, room_pref):
    free_groups = list(group_pref.keys())
    group_match, room_match = {}, {}
    next_proposal = {g: 0 for g in free_groups}

    # room_rank[room][group] gives smaller numbers to more preferred groups
    room_rank = {r: {g: i for i, g in enumerate(prefs)}
                  for r, prefs in room_pref.items()}

    while free_groups: #The room is already matched with another group.

        group = free_groups.pop(0)
        room = group_pref[group][next_proposal[group]]
        next_proposal[group] += 1
        #Case 1: The room is free. The room accepts the proposal because it isn't matched with anyone.
        if room not in room_match:
            room_match[room] = group
            group_match[group] = room
        #Case 2: The room is already matched with another group.
        Else:
            occupied_current = room_match[room]
            #If the new group has higher priority than the room switches to the new group.
            if room_rank[room][group] < room_rank[room][current]:
                room_match[room] = group # room keeps preferred group
                group_match[group] = room
                del group_match[current]
                free_groups.append(current)
            #If not, reject the new group proposal.
            else:
                free_groups.append(group) # rejected; tries next room later
    return group_match
```

Test plan and results

Testing used deterministic preference lists. Each case compares actual output with the expected stable matching and also checks stability by searching for blocking pairs.

Case	Input summary	Expected	Actual	Result
Given 3x3	Groups A-C and Rooms 1-3 from project prompt.	A->Room1, B->Room2, C->Room3	A->Room1, B->Room2, C->Room3	Pass; stable
Conflict-heavy	All groups first choose R1; room priorities force rejections.	C->R1, B->R2, A->R3	C->R1, B->R2, A->R3	Pass; stable
Mixed 4x4	Four groups and four rooms with crossed preferences.	G1->R2, G2->R3, G3->R1, G4->R4	G1->R2, G2->R3, G3->R1, G4->R4	Pass; stable

Runtime and memory measurements

Methodology: Python 3 timing used time.perf_counter(); memory used tracemalloc peak allocation. Representative synthetic data used equal numbers of groups and rooms with complete randomized preference lists and a fixed random seed. Measurements are from one local run.

n groups / rooms	Time (ms)	Peak memory (KiB)	Matched
10	0.07	3.9	10
100	1.85	343.6	100
500	106.09	9769.4	500
1000	533.90	45886.6	1000

Theoretical complexity is $O(n^2)$ proposals in the worst case for n groups and n rooms. The ranking dictionary also requires $O(n^2)$ memory because it stores each room's ranking of every group. The observed results grow consistently with this expectation, especially for the 500-and 1000-pair cases.

Trade-offs, limitations, and future work

This solution is simple, deterministic, and guarantees stable assignments, but the group-proposing version favors groups over rooms when multiple stable matchings exist. It assumes equal numbers of groups and rooms, complete preference lists, and no room capacities greater than one. It also treats preferences as strict rankings, so ties are not supported. Future work could add group sizes and room capacities, support incomplete lists and ties, allow priority weights, and replace `pop(0)` with a queue for cleaner large-scale performance.